



CSC462 – ARTIFICIAL INTELLIGENCE

Lab Assignment 1 Report



Submitted By:

Muhammad Rizwan Shafiq
SP24-BCS-069
BCS-4B

Submitted To:

Dr. Muhammad Imran

Lab 2: Python Collections

This lab demonstrates Python's built-in data structures including lists and dictionaries through multiple small programming exercises.

Task 1: Merging and Sorting Lists

The task merges two lists provided by the user and sorts them in ascending order.

Code:

```
list1 = list(map(int, input("Enter numbers for first list (separated by spaces): ").split()))
list2 = list(map(int, input("Enter numbers for second list (separated by spaces): ").split()))
merged_list = list1 + list2
merged_list.sort()
print("Merged and Sorted List:", merged_list)
```

Task 2: Finding Smallest and Largest Elements

This task extends the previous one by identifying the smallest and largest element from the merged list.

Code:

```
merged_list.sort() # from lab task 1
print("Smallest Element:", min(merged_list))
print("Largest Element:", max(merged_list))
```

Task 3: Finite Difference Derivative of $\sin(x)$

This program calculates the derivative of $\sin(x)$ using finite differences for different values of h and compares it with the analytical derivative $\cos(x)$.

Code:

```
# Task 3: finite-difference derivative of sin(x)
from math import sin, cos, pi

def forward_diff_sin(x, h):
    return (sin(x + h) - sin(x)) / h

def run(h=0.001, dx=0.001):
    xs = []
    x = -pi
    # build x from -pi to pi (inclusive) with increment dx
    while x <= pi + 1e-12:
```

```

        xs.append(x)
        x += dx

    approx = [forward_diff_sin(x, h) for x in xs]
    truth = [cos(x) for x in xs]
    errors = [abs(a - t) for a, t in zip(approx, truth)]

    # Print a few sample rows
    print(f"\n=== Results with h = {h} (grid dx = {dx}) ===")
    print("    x\t\t approx d/dx sin(x)\t cos(x)\t\t abs error")
    for i in range(0, len(xs), max(1, len(xs)//8)): # ~8 samples across
the interval
        print(f"{xs[i]: .6f}\t {approx[i]: .8f}\t {truth[i]: .8f}\t
{errors[i]: .2e}")

    print(f"\nMax abs error: {max(errors):.2e}")
    print(f"Mean abs error: {sum(errors)/len(errors):.2e}")

# Run the experiment for three h values
run(h=0.001)
run(h=0.01)
run(h=0.1)

```

Task 4: Birthday Dictionary

A dictionary is created to store names and birthdays, allowing the user to retrieve a person's birthday by entering their name.

Code:

```

# Dictionary of names and birthdays
birthdays = {
    "Albert Einstein": "03/14/1879",
    "Benjamin Franklin": "01/17/1706",
    "Ada Lovelace": "12/10/1815"
}

print("Welcome to the birthday dictionary. We know the birthdays of:")
for name in birthdays:
    print(name)

# Asking user for a name
person = input("Who's birthday do you want to look up? ")

# Displaying result

```

```
if person in birthdays:
    print(f"{person}'s birthday is {birthdays[person]}.")
else:
    print("Sorry, we don't have that person's birthday in the dictionary.")
```

Task 5: Extracting Specific Keys from a Dictionary

This program extracts specific key-value pairs from a given dictionary using dictionary comprehension.

Code:

```
# Given dictionary:
sample_dict = { "name": "Kelly",
               "age": 25, "salary": 8000, "city": "New york"}

# # Keys to extract
keys = ["name", "salary"]

# Expected output:
# {'name': 'Kelly', 'salary': 8000}

# Extracting dictionary using dictionary comprehension
new_dict = {k: sample_dict[k] for k in keys}
print(new_dict)
```

Lab 3: Breadth First Search (BFS)

This lab focuses on implementing the Breadth-First Search algorithm for two different problems: a city navigation problem and a maze pathfinding problem.

Task 1: Romania Map Pathfinding

The BFS algorithm is applied to find the shortest path (in terms of edges) between cities in the Romania map, specifically from Arad to Bucharest.

Code:

```
from collections import deque
from typing import Dict, List, Tuple, Optional

Graph = Dict[str, List[Tuple[str, int]]]

romania: Graph = {
    "Arad":      [("Zerind", 75), ("Timisoara", 118), ("Sibiu", 140)],
    "Zerind":    [("Arad", 75), ("Oradea", 71)],
    "Oradea":    [("Zerind", 71), ("Sibiu", 151)],
```

```

    "Sibiu":      [("Arad", 140), ("Oradea", 151), ("Fagaras", 99),
("Rimnicu Vilcea", 80)],
    "Timisoara":  [("Arad", 118), ("Lugoj", 111)],
    "Lugoj":      [("Timisoara", 111), ("Mehadia", 70)],
    "Mehadia":    [("Lugoj", 70), ("Drobeta", 75)],
    "Drobeta":    [("Mehadia", 75), ("Craiova", 120)],
    "Craiova":    [("Drobeta", 120), ("Rimnicu Vilcea", 146), ("Pitesti",
138)],
    "Rimnicu Vilcea": [("Sibiu", 80), ("Craiova", 146), ("Pitesti", 97)],
    "Fagaras":    [("Sibiu", 99), ("Bucharest", 211)],
    "Pitesti":    [("Rimnicu Vilcea", 97), ("Craiova", 138), ("Bucharest",
101)],
    "Bucharest":  [("Fagaras", 211), ("Pitesti", 101), ("Giurgiu", 90),
("Urziceni", 85)],
    "Giurgiu":    [("Bucharest", 90)],
    "Urziceni":   [("Bucharest", 85), ("Hirsova", 98), ("Vaslui", 142)],
    "Hirsova":    [("Urziceni", 98), ("Eforie", 86)],
    "Eforie":     [("Hirsova", 86)],
    "Vaslui":     [("Urziceni", 142), ("Iasi", 92)],
    "Iasi":       [("Vaslui", 92), ("Neamt", 87)],
    "Neamt":      [("Iasi", 87)],
}

```

```

def bfs_path(graph: Graph, start: str, goal: str) -> Optional[List[str]]:
    if start == goal:
        return [start]

    visited = set([start])
    parent: Dict[str, Optional[str]] = {start: None}
    q = deque([start])

    while q:
        u = q.popleft()
        for v, _w in graph.get(u, []): # ignore weights in BFS
            if v not in visited:
                visited.add(v)
                parent[v] = u
                if v == goal:
                    path = [v]
                    while parent[path[-1]] is not None:
                        path.append(parent[path[-1]])
                    path.reverse()
                    return path
                q.append(v)
    return None

```

```
def path_distance(graph: Graph, path: List[str]) -> int:
    total = 0
    for u, v in zip(path, path[1:]):
        for nbr, d in graph[u]:
            if nbr == v:
                total += d
                break
        else:
            raise ValueError(f"No road between {u} and {v}")
    return total

if __name__ == "__main__":
    start, goal = "Arad", "Bucharest"
    path = bfs_path(romania, start, goal)
    if path is None:
        print(f"No path found from {start} to {goal}.")
    else:
        dist = path_distance(romania, path)
        print("BFS path (fewest cities):", " -> ".join(path))
        print("Total distance on that path:", dist, "km")
```

Task 2: BFS Maze Traversal

In this task, BFS is implemented to find a path in a 6x6 maze grid between a start and goal position. It reports the sequence of moves required to reach the goal.

Code:

```
# ----- Task 2 (BFS Maze) -----

from collections import deque

# ----- Node definition (as in Lab 3) -----
class Node:
    def __init__(self, state, parent=None, actions=None, total_cost=0):
        self.state = state          # here: (x, y) grid coordinate or
        goal's coordinate
        self.parent = parent        # parent Node for backtracking
        self.actions = actions or [] # list of neighbor states
        self.total_cost = total_cost

# ----- Build the maze graph to match the figure -----
# Coordinates: x = 0..5 left→right, y = 0..5 top→bottom
# The goal is above (4,0), so we make it at (4, -1) and link it to (4,0).
```

```

START = (2, 2)
GOAL  = (4, -1)

def make_maze_graph():
    graph = {}

    def ensure(state):
        if state not in graph:
            graph[state] = Node(state)
        return graph[state]

    def connect(a, b):
        A, B = ensure(a), ensure(b)
        if b not in A.actions:
            A.actions.append(b)
        if a not in B.actions:
            B.actions.append(a)

    # Outer ring (square)
    for x in range(5):
        connect((x, 0), (x + 1, 0)) # top edge
    for y in range(5):
        connect((5, y), (5, y + 1)) # right edge
    for x in range(5):
        connect((x, 5), (x + 1, 5)) # bottom edge
    for y in range(5):
        connect((0, y), (0, y + 1)) # left edge

    # Vertical corridor under the start
    connect((2, 2), (2, 3))
    connect((2, 3), (2, 4))
    connect((2, 4), (2, 5))

    # Right-side extras from the figure
    connect((4, 2), (5, 2)) # small dead-end in the middle
    right
    connect((4, 5), (4, 4)) # bottom-right 2x2 block
    connect((4, 4), (5, 4))
    connect((4, 5), (5, 5))

    # Goal node above the top row at column x=4
    connect(GOAL, (4, 0))

    # Make sure START exists

```

```

    ensure(START)

    return graph

# ----- BFS exactly as in Lab 3 -----
def bfs(graph, start, goal):
    # reset parents
    for node in graph.values():
        node.parent = None

    frontier = deque([start])
    visited = {start}
    graph[start].parent = None

    while frontier:
        current = frontier.popleft()
        if current == goal:
            break
        for child in graph[current].actions:
            if child not in visited:
                visited.add(child)
                graph[child].parent = graph[current]
                frontier.append(child)

    # Reconstruct path (states) from goal back to start
    if graph.get(goal) is None or graph[goal].parent is None and goal != start:
        return None # no path

    path = []
    node = graph[goal]
    while node is not None:
        path.append(node.state)
        node = node.parent
    path.reverse()
    return path

# ----- Convert a path of coordinates to U/D/L/R moves -----
def to_moves(path):
    moves = []
    for (x1, y1), (x2, y2) in zip(path, path[1:]):
        if x2 == x1 and y2 == y1 + 1:
            moves.append("D")
        elif x2 == x1 and y2 == y1 - 1:
            moves.append("U")

```



```
        elif y2 == y1 and x2 == x1 + 1:
            moves.append("R")
        elif y2 == y1 and x2 == x1 - 1:
            moves.append("L")
        else:
            moves.append("?")
    return moves

graph = make_maze_graph()
path = bfs(graph, START, GOAL)
moves = to_moves(path) if path else None

print("Path (states):", path)
print("Moves:", moves)
print("Steps:", len(moves) if moves else 0)
```

Expected Output:

Path (states): [(2,2), (2,3), (2,4), (2,5), (3,5), (4,5), (4,4), (4,3), (4,2), (4,1), (4,0), (4,-1)]

Moves: ['D', 'D', 'D', 'R', 'R', 'U', 'U', 'U', 'U']

Steps: 9

Conclusion

Through Lab 2 and Lab 3, the fundamental understanding of Python collections and graph traversal algorithms (BFS) was developed. The exercises demonstrated practical applications of lists, dictionaries, and search algorithms in solving computational problems.